

```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')
```

✓ Missing and Duplicate Data

```
data = {
    "Name": ["Ali", "Sara", "Ahmed", "Fatima", "Ali", "Usman", "Zara"],
    "Age": [20, 22, None, 21, 20, None, 23],
    "City": ["Lahore", "Karachi", "Lahore", None, "Lahore", "Islamabad", "Karachi"],
    "Marks": [85, 90, 78, 88, 85, None, 91]
}

df = pd.DataFrame(data)
df
```

	Name	Age	City	Marks
0	Ali	20.0	Lahore	85.0
1	Sara	22.0	Karachi	90.0
2	Ahmed	NaN	Lahore	78.0
3	Fatima	21.0	None	88.0
4	Ali	20.0	Lahore	85.0
5	Usman	NaN	Islamabad	NaN
6	Zara	23.0	Karachi	91.0

```
len(df)
```

```
7
```

```
df.isnull().sum()
```

	0
Name	0
Age	2
City	1
Marks	1

dtype: int64

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
#
```

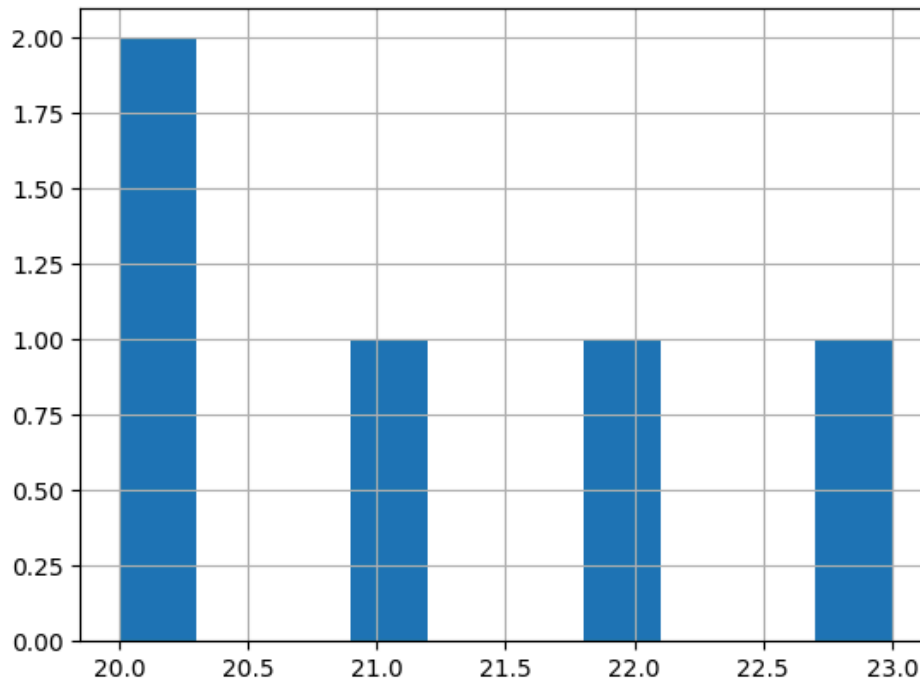
```

---  ---  ---  ---
0   Name    7 non-null    object
1   Age     5 non-null    float64
2   City    6 non-null    object
3   Marks   6 non-null    float64
dtypes: float64(2), object(2)
memory usage: 356.0+ bytes

```

```
df['Age'].hist()
```

<Axes: >



or

```
print(df['Age'].skew())
```

```
0.5413870509510876
```

```

if abs(df['Age'].skew()) < 0.5:
    df['Age'].fillna(df['Age'].mean(), inplace=True)
else:
    df['Age'].fillna(df['Age'].median(), inplace=True)

```

```
df['Marks'].skew()
```

```
np.float64(-1.070612154025652)
```

```

if abs(df['Marks'].skew()) < 0.5:
    df['Marks'].fillna(df['Marks'].mean(), inplace=True)
else:
    df['Marks'].fillna(df['Marks'].median(), inplace=True)

```

```
df['City'].fillna(df['City'].mode()[0], inplace=True)
```

df

	Name	Age	City	Marks
0	Ali	20.0	Lahore	85.0
1	Sara	22.0	Karachi	90.0
2	Ahmed	21.0	Lahore	78.0
3	Fatima	21.0	Lahore	88.0
4	Ali	20.0	Lahore	85.0
5	Usman	21.0	Islamabad	86.5
6	Zara	23.0	Karachi	91.0

```
print("Duplicates:", df.duplicated().sum())
```

Duplicates: 1

```
df.drop_duplicates(inplace=True)
df
```

	Name	Age	City	Marks
0	Ali	20.0	Lahore	85.0
1	Sara	22.0	Karachi	90.0
2	Ahmed	21.0	Lahore	78.0
3	Fatima	21.0	Lahore	88.0
5	Usman	21.0	Islamabad	86.5
6	Zara	23.0	Karachi	91.0

✓ Data Type Conversion

```
data2 = {
    "Product": ["Laptop", "Phone", "Tablet", "Monitor"],
    "Price": ["1200", "800", "not available", "300"],
    "Quantity": ["2", "5", "3", "two"],
    "Date": ["2023-01-05", "2023/02/10", "2023-04-01", "invalid_date"]
}
```

```
df2 = pd.DataFrame(data2)
df2
```

	Product	Price	Quantity	Date
0	Laptop	1200	2	2023-01-05
1	Phone	800	5	2023/02/10
2	Tablet	not available	3	2023-04-01
3	Monitor	300	two	invalid_date

```
df2.dtypes
```

```
      0
Product  object
Price    object
Quantity object
Date     object

dtype: object
```

```
df2['Price'] = pd.to_numeric(df2['Price'], errors='coerce')
df2['Quantity'] = pd.to_numeric(df2['Quantity'], errors='coerce')
df2['Date'] = pd.to_datetime(df2['Date'], errors='coerce')
```

```
df2
```

	Product	Price	Quantity	Date
0	Laptop	1200.0	2.0	2023-01-05
1	Phone	800.0	5.0	NaT
2	Tablet	NaN	3.0	2023-04-01
3	Monitor	300.0	NaN	NaT

```
df2.dtypes
```

```
      0
Product      object
Price      float64
Quantity    float64
Date  datetime64[ns]

dtype: object
```

✓ Feature Scaling

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
```

```
X = np.array([[150], [160], [170], [180], [190]])
```

✓ Normalization

Normalization rescales data to a fixed range, typically between 0 and 1. It preserves the shape of the original distribution but compresses the scale. Normalization is useful when:

- Data does not follow a normal distribution
- You are using distance-based algorithms (k-NN, k-means)
- Feature bounds are known and meaningful

```
X_new = (X - X_min) / (X_max - X_min)
```

```
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)

print(X_normalized)
```

```
[[0.  ]
 [0.25]
 [0.5  ]
 [0.75]
 [1.  ]]
```

✓ Standardization

Standardization transforms data to have:

- Mean = 0
- Standard deviation = 1

It does not restrict values to a fixed range and works well when data follows a normal distribution.

Standardization is preferred when:

- Data contains outliers
- Features follow a Gaussian distribution
- Using algorithms like linear regression, logistic regression, SVM, or PCA

```
X_new = (X - mean) / Std
```

```
scaler = StandardScaler()
X_standardized = scaler.fit_transform(X)

print(X_standardized)
```

```
[[-1.41421356]
 [-0.70710678]
 [ 0.         ]
 [ 0.70710678]
 [ 1.41421356]]
```

The choice between normalization and standardization depends mainly on the distribution of the data and the presence of skewness. If a feature is approximately normally distributed (symmetric and bell-shaped), standardization is preferred because it centers the data around zero and scales it based on variance. If the data is not normally distributed or has clear bounds, normalization is often more suitable, especially for distance-based algorithms.

Skewness is a key indicator in this decision. Skewness measures the asymmetry of a distribution and can be visually checked using a histogram or numerically computed using the `.skew()` function. A skewness

value close to 0 indicates a symmetric distribution. In practice, if skewness lies roughly between -0.5 and $+0.5$, the data can be treated as approximately normal and standardization is recommended. If skewness is greater than $+1$ or less than -1 , the distribution is highly skewed, and normalization is generally preferred unless the data is transformed first (e.g., log transformation).

```
from sklearn.model_selection import train_test_split

X = np.array([[10], [20], [30], [40], [50]])

X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Train:\n", X_train_scaled)
print("Test:\n", X_test_scaled)
```

```
Train:
[[ 1.18321596]
 [-0.16903085]
 [-1.52127766]
 [ 0.50709255]]
Test:
[[-0.84515425]]
```

✓ Encoding

Encoding is a feature engineering technique used to convert categorical variables into numerical form. Since most machine learning algorithms operate only on numerical data, categorical features such as colors, countries, or severity levels must be encoded before model training.

The choice of encoding method depends on whether the categorical data has no order (nominal) or a meaningful order (ordinal).

```
data3 = {
    "Name": ["Ali", "Sara", "Ahmed", "Zara", "Usman"],
    "Department": ["HR", "IT", "Finance", "IT", "HR"],
    "Gender": ["Male", "Female", "Male", "Female", "Male"],
    "Performance": ["Low", "High", "Medium", "High", "Low"]
}

df3 = pd.DataFrame(data3)
df3
```

	Name	Department	Gender	Performance
0	Ali	HR	Male	Low
1	Sara	IT	Female	High
2	Ahmed	Finance	Male	Medium
3	Zara	IT	Female	High
4	Usman	HR	Male	Low

✓ Label Encoding

Label Encoding converts each category into a unique integer value. The entire categorical column is replaced by these numeric labels.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df3['Gender'] = le.fit_transform(df3['Gender'])

df3
```

	Name	Department	Gender	Performance
0	Ali	HR	1	Low
1	Sara	IT	0	High
2	Ahmed	Finance	1	Medium
3	Zara	IT	0	High
4	Usman	HR	1	Low

✓ One Hot Encoding

One-Hot Encoding converts a categorical feature into multiple binary columns, one for each unique category. A value of 1 indicates the presence of that category, while 0 indicates absence.

```
one_hot = pd.get_dummies(df3['Department'])
one_hot
```

	Finance	HR	IT
0	False	True	False
1	False	False	True
2	True	False	False
3	False	False	True
4	False	True	False

Ordinal Encoding

Ordinal Encoding assigns numerical values to categories based on a predefined order. Unlike label encoding, the order is explicitly defined by the user, making it safer and more meaningful for ordinal data.

```
from sklearn.preprocessing import OrdinalEncoder

encoder = OrdinalEncoder(categories=[['Low', 'Medium', 'High']])
df3['Performance_encoded'] = encoder.fit_transform(df3[['Performance']])

df3
```

	Name	Department	Gender	Performance	Performance_encoded
0	Ali	HR	1	Low	0.0
1	Sara	IT	0	High	2.0
2	Ahmed	Finance	1	Medium	1.0
3	Zara	IT	0	High	2.0
4	Usman	HR	1	Low	0.0

```
train = pd.Series(['Low', 'Medium', 'High'])
test = pd.Series(['Medium', 'Low'])

encoder = LabelEncoder()
train_encoded = encoder.fit_transform(train)

test_encoded = encoder.transform(test)

print("Train:", train_encoded)
print("Test:", test_encoded)
```

Selecting Multiple columns together

```
data = {
    "Name": ["Ali", "Sara", "Ahmed"],
    "Age": [20, 22, 21],
    "Salary": [50000.0, 60000.5, 55000.2],
    "City": ["Lahore", "Karachi", "Islamabad"],
    "Is_Student": [True, False, True]
}

df = pd.DataFrame(data)
```

```
df.dtypes
```

```
0
Name      object
Age       int64
Salary    float64
City      object
Is_Student bool
```

dtype: object

```
df.select_dtypes(include=['int64', 'float64'])# Numerical columns
```

```
Age Salary
0    20  50000.0
1    22  60000.5
2    21  55000.2
```

```
df.select_dtypes(exclude=['int64', 'float64']) #exclude numerical columns
```

```
Name      City  Is_Student
0      Ali    Lahore      True
1     Sara   Karachi    False
2   Ahmed  Islamabad    True
```

```
df.select_dtypes(include=['int64'])#select integer columns
```

```
Age
0    20
1    22
2    21
```

```
df.select_dtypes(include=['float64'])# select float columns
```

```
Salary
0  50000.0
1  60000.5
2  55000.2
```

```
df.select_dtypes(include=['object'])#Select Categorical Columns (object/string)
```

	Name	City
0	Ali	Lahore
1	Sara	Karachi
2	Ahmed	Islamabad

```
df.select_dtypes(include=['bool'])#select boolean columns
```

	Is_Student
0	True
1	False
2	True

```
df.select_dtypes(include=['int64', 'float64'])#multiple types together
```

Start coding or [generate](#) with AI.